

Banco de ejercicios tipo examen

Patrones detectados en el taller DVD Rental · LEGO · Employees — 15 ejercicios con solución verificada, código y justificación

PostgreSQL 16 · dvdrental (Pagila) · lego (Rebrickable) · employees · todos los resultados y planes fueron ejecutados y comprobados

Cómo se construyó este banco. Cada ejercicio del taller original combina un patrón de SQL con una intención de examen: filtrar, agrupar, encadenar tablas, o justificar un índice con evidencia. Este documento aísla esos patrones y los reutiliza sobre datos distintos, para que sirvan de práctica real y no de memorización del taller. Cada enunciado indica el **patrón** que evalúa; cada solución incluye el código, el resultado real y un bloque "Por qué" que explica la decisión, no sólo la sintaxis.

Ejercicio 1

patrón: filtros combinados + JOIN

Películas de acción con alquiler flexible

Sobre **dvdrental**, escribe una consulta que liste las películas de la categoría 'Action' cuya duración de alquiler (rental_duration) sea de al menos 5 días. Muestra film_id, title, rental_duration y rental_rate, ordenadas por tarifa descendente y luego por título. Limita a 5 filas.

SOLUCIÓN

```
SELECT f.film_id, f.title, f.rental_duration, f.rental_rate
FROM film f
JOIN film_category fc ON fc.film_id = f.film_id
JOIN category c      ON c.category_id = fc.category_id
WHERE c.name = 'Action' AND f.rental_duration >= 5
ORDER BY f.rental_rate DESC, f.title
LIMIT 5;
```

film_id	title	rental_duration	rental_rate
212	Darn Forrester	7	4.99
229	Devil Desire	6	4.99
253	Drifter Commandments	5	4.99
271	Easy Gladiator	5	4.99
371	Gosford Donnie	5	4.99

POR QUÉ

category no es una columna de film: es una tabla aparte, unida a través de film_category, una tabla puente que resuelve una relación muchos-a-muchos (una película puede tener varias categorías y viceversa). Por eso hacen falta **dos** JOIN para llegar del nombre de la categoría hasta la película, no uno.

El WHERE combina una condición de texto sobre la tabla unida (c.name) con una condición numérica sobre la tabla original (f.rental_duration): el motor puede aplicar ambas en cualquier orden porque las evalúa después de resolver los JOIN, no importa en qué línea se escriban.

Clasificar películas por duración

Sobre **dvdrental**, clasifica cada película en tres categorías según su duración: 'Corta (<90 min)', 'Media (90-120 min)' y 'Larga (>120 min)'. Muestra cuántas películas hay en cada categoría y su tarifa de alquiler promedio.

SOLUCIÓN

```
SELECT CASE
  WHEN length < 90 THEN 'Corta (<90 min)'
  WHEN length BETWEEN 90 AND 120 THEN 'Media (90-120 min)'
  ELSE 'Larga (>120 min)'
END AS categoria_duracion,
COUNT(*) AS n_peliculas,
ROUND(AVG(rental_rate),2) AS tarifa_prom
FROM film
GROUP BY categoria_duracion
ORDER BY n_peliculas DESC;
```

categoria_duracion	n_peliculas	tarifa_prom
Larga (>120 min)	457	3.03
Corta (<90 min)	320	2.94
Media (90-120 min)	223	2.94

POR QUÉ

El CASE dentro del GROUP BY funciona porque PostgreSQL permite agrupar por una expresión con el mismo texto que aparece en el SELECT, no sólo por columnas físicas. El motor primero calcula la expresión fila por fila y después agrupa por su resultado, exactamente igual que si esa expresión fuera una columna.

Un error frecuente es escribir GROUP BY length pensando que basta con agrupar por la columna original: eso produciría un grupo por cada valor distinto de duración (decenas de grupos), no por las tres categorías del enunciado.

Cientes muy por encima del gasto típico

Sobre **dvdrental**, encuentra los clientes cuyo **total pagado** supera en 10 veces el gasto promedio por cliente (no el promedio de cada pago individual, sino el promedio de los totales agrupados por cliente). Muestra `customer_id` y `total_pagado`, ordenados de mayor a menor, limitado a 5.

SOLUCIÓN

```
SELECT customer_id, total_pagado
FROM (
  SELECT customer_id, SUM(amount) AS total_pagado
  FROM payment
  GROUP BY customer_id
) t
WHERE total_pagado > (
  SELECT AVG(total) FROM (
    SELECT customer_id, SUM(amount) AS total
    FROM payment GROUP BY customer_id
  ) prom
) * 10
ORDER BY total_pagado DESC
LIMIT 5;
```

customer_id	total_pagado
148	211.55
526	208.58
178	194.61
137	191.62
144	189.60

Trampa típica de examen. Si en lugar de la subconsulta se usa `AVG(amount)` directamente sobre `payment`, se está promediando cada *pago individual* (599 pagos, promedio $\approx 4,20$) en vez del *total por cliente* (599 clientes, promedio $\approx 102,36$). Son números completamente distintos y responden a preguntas distintas: "¿cuánto vale un pago típico?" contra "¿cuánto gasta un cliente típico en total?". El enunciado pide lo segundo, por eso la subconsulta agrupa primero.

POR QUÉ

Esta consulta anida **tres** niveles: la subconsulta más interna agrupa pagos por cliente; la del medio promedia esos totales; la externa filtra contra ese promedio multiplicado por 10. Cada nivel resuelve una unidad de análisis distinta (pago → cliente → toda la base), y mezclar los niveles es el error más común con subconsultas de agregación.

Una alternativa más legible sería usar dos CTE (`WITH totales AS ( . . . )`) en vez de subconsultas anidadas; el resultado es idéntico, cambia sólo la forma de escribirlo.

Parejas de actores que coincidieron en varias películas

Sobre **dvdbrental**, encuentra las parejas de actores que aparecen juntos en 3 o más películas. No debe aparecer la misma pareja dos veces (ni un actor emparejado consigo mismo). Muestra ambos nombres y el número de películas compartidas, de mayor a menor, limitado a 5.

SOLUCIÓN

```
SELECT a1.first_name || ' ' || a1.last_name AS actor1,
       a2.first_name || ' ' || a2.last_name AS actor2,
       COUNT(*) AS pelis_juntas
FROM film_actor fa1
JOIN film_actor fa2 ON fa1.film_id = fa2.film_id
                    AND fa1.actor_id < fa2.actor_id  -- evita duplicados e inversos
JOIN actor a1 ON a1.actor_id = fa1.actor_id
JOIN actor a2 ON a2.actor_id = fa2.actor_id
GROUP BY a1.actor_id, a2.actor_id, actor1, actor2
HAVING COUNT(*) >= 3
ORDER BY pelis_juntas DESC
LIMIT 5;
```

actor1	actor2	pelis_juntas
Julia Mcqueen	Henry Berry	7
Morgan Mcdormand	Will Wilson	6
Val Bolger	Rock Dukakis	6
Kirsten Paltrow	Warren Nolte	6
Cuba Olivier	Mary Keitel	6

POR QUÉ

Un **self-join** es un JOIN de una tabla contra sí misma: aquí film_actor se une consigo misma usando alias distintos (fa1, fa2) para representar dos roles distintos del mismo dato — "el primer actor de la pareja" y "el segundo". Sin la condición fa1.actor_id < fa2.actor_id, cada pareja aparecería **cuatro** veces: (A,B), (B,A), y además cada actor emparejado consigo mismo (A,A) y (B,B).

El operador < en vez de != resuelve dos problemas a la vez: excluye los pares de un actor consigo mismo **y** fuerza un orden fijo (el id menor siempre va primero), lo que elimina el duplicado invertido sin necesitar un DISTINCT adicional.

Películas que nunca se han alquilado

Sobre **dvdrental**, lista las películas del catálogo que jamás han sido alquiladas (ni siquiera tienen copias con historial de alquiler). Muestra cuántas son en total y, además, las primeras 10 por orden alfabético.

SOLUCIÓN

```
SELECT f.film_id, f.title
FROM film f
LEFT JOIN inventory i ON i.film_id = f.film_id
LEFT JOIN rental r    ON r.inventory_id = i.inventory_id
WHERE r.rental_id IS NULL
ORDER BY f.title
LIMIT 10;

-- conteo total
SELECT COUNT(*) AS total_sin_alquiler
FROM film f
LEFT JOIN inventory i ON i.film_id = f.film_id
LEFT JOIN rental r    ON r.inventory_id = i.inventory_id
WHERE r.rental_id IS NULL;
```

film_id	title
1	Academy Dinosaur
14	Alice Fantasia
33	Apollo Teen
36	Argonauts Town
38	Ark Ridgemon

5 de 10 filas mostradas · total_sin_alquiler = 43

POR QUÉ

Este patrón se llama **anti-join**: buscar filas de la tabla izquierda que *no tienen* correspondencia en la derecha. El truco es usar `LEFT JOIN` — que conserva todas las filas de `film` aunque no encuentren pareja — y después filtrar por `IS NULL` sobre una columna de la tabla derecha que nunca sería nula si el `JOIN` hubiera encontrado algo.

Hacen falta dos `LEFT JOIN` encadenados porque la relación no es directa: `film` no tiene una columna que apunte a `rental`; hay que pasar primero por `inventory` (las copias físicas). Un error común es usar `INNER JOIN` en el segundo salto: eso descartaría las películas sin ninguna copia en inventario antes de llegar a comprobar si tienen alquileres, dando un resultado distinto (e incorrecto para "nunca alquilada").

Versatilidad de un actor por número de categorías distintas

Sobre **dvdrental**, para cada actor calcula en cuántas **categorías distintas** de película ha actuado y en cuántas películas en total. Ordena por número de categorías descendente y muestra los 5 actores más versátiles.

SOLUCIÓN

```
SELECT a.first_name || ' ' || a.last_name AS actor,
       COUNT(DISTINCT c.category_id) AS categorías,
       COUNT(DISTINCT f.film_id)     AS películas
FROM actor a
JOIN film_actor fa    ON fa.actor_id = a.actor_id
JOIN film f          ON f.film_id = fa.film_id
JOIN film_category fc ON fc.film_id = f.film_id
JOIN category c      ON c.category_id = fc.category_id
GROUP BY a.actor_id, actor
ORDER BY categorías DESC, películas DESC
LIMIT 5;
```

actor	categorías	películas
Gina Degeneres	16	42
Uma Wood	16	35
Groucho Dunst	16	35
Kevin Garland	16	33
Ewan Gooding	16	33

POR QUÉ

El COUNT(DISTINCT ...) es obligatorio y no un capricho de estilo: al atravesar cuatro JOIN, cada fila del resultado representa una combinación única de (actor, película, categoría). Un actor con 35 películas y varias categorías por película puede generar más de 35 filas; contar sin DISTINCT inflaría el número de películas.

Nótese que hay **dos** COUNT(DISTINCT ...) sobre columnas diferentes en el mismo SELECT: eso es válido y frecuente quando se necesitan dos métricas de granularidad distinta sobre el mismo conjunto de filas ya unido.

Los 3 mejores clientes de cada tienda

Sobre **dvdrental**, identifica los 3 clientes que más han pagado **en cada tienda** (la tienda se determina por el empleado —staff— que procesó el pago). Muestra tienda, cliente, total pagado y su posición dentro de la tienda.

SOLUCIÓN

```
SELECT store, customer_id, total, rnk
FROM (
  SELECT s.store_id AS store,
         c.customer_id,
         SUM(p.amount) AS total,
         RANK() OVER (PARTITION BY s.store_id
                     ORDER BY SUM(p.amount) DESC) AS rnk
  FROM payment p
  JOIN customer c ON c.customer_id = p.customer_id
  JOIN staff s   ON s.staff_id = p.staff_id
  GROUP BY s.store_id, c.customer_id
) x
WHERE rnk <= 3
ORDER BY store, rnk;
```

store	customer_id	total	rnk
1	137	112.78	1
1	176	111.78	2
1	178	108.81	3
2	187	110.81	1
2	148	110.78	2
2	211	108.77	3

POR QUÉ

RANK() OVER (PARTITION BY ... ORDER BY ...) no colapsa filas como GROUP BY: calcula una posición *dentro de cada partición* sin perder el detalle de cada fila. Por eso hace falta envolver la consulta en una subconsulta (o un CTE): el filtro WHERE rnk <= 3 no puede ir en la misma consulta donde se calcula rnk, porque las funciones de ventana se evalúan *después* del WHERE en el orden lógico de ejecución de SQL.

Se usó RANK() y no ROW_NUMBER() a propósito: si dos clientes empatan en el tercer lugar, RANK() les da a ambos el puesto 3 (y salta el 4), mientras que ROW_NUMBER() forzaría un desempate arbitrario. Para "los 3 mejores" con empates reales, RANK() es la elección honesta.

Temas grandes y con sets grandes

Sobre **lego**, encuentra los temas que cumplen **ambas** condiciones a la vez: tienen más de 30 sets publicados y sus sets tienen, en promedio, más de 300 piezas. Ordena por piezas promedio descendente.

SOLUCIÓN

```
SELECT t.name AS tema,
       COUNT(*) AS n_sets,
       ROUND(AVG(s.num_parts),1) AS piezas_prom
FROM lego_sets s
JOIN lego_themes t ON t.id = s.theme_id
GROUP BY t.name
HAVING COUNT(*) > 30 AND AVG(s.num_parts) > 300
ORDER BY piezas_prom DESC;
```

tema	n_sets	piezas_prom
Star Wars Episode 4/5/6	148	592.5
Educational and Dacta	33	473.2
Ferrari	32	420.2
Universal Building Set	34	417.3
Basic Set	257	390.7

POR QUÉ

HAVING filtra *grupos ya formados*, mientras que WHERE filtra filas *antes* de agrupar. Aquí ambas condiciones —COUNT(*) > 30 y AVG(num_parts) > 300— dependen de agregaciones, así que no existe forma de expresarlas con WHERE: el motor todavía no sabe cuántas filas tendrá cada grupo hasta haberlas agrupado.

Combinar dos condiciones de HAVING con AND funciona igual que en WHERE: ambas deben cumplirse. Un error de examen frecuente es escribir dos cláusulas HAVING separadas — SQL sólo permite una por consulta.

Cuántos subtemas tiene cada tema padre

Sobre **lego**, la tabla `lego_themes` es jerárquica: cada tema puede tener un `parent_id` que apunta a otro tema de la misma tabla. Cuenta cuántos subtemas directos tiene cada tema padre, mostrando sólo los 10 con más subtemas.

SOLUCIÓN

```
SELECT p.name AS tema_padre,
       COUNT(h.id) AS n_subtemas
FROM   lego_themes p
JOIN   lego_themes h ON h.parent_id = p.id   -- self-join: h = hijo, p = padre
GROUP BY p.name
ORDER BY n_subtemas DESC
LIMIT 10;
```

tema_padre	n_subtemas
Bionicle	38
Collectible Minifigures	24
Town	20
Educational and Dacta	20
Space	20

POR QUÉ

Es el mismo mecanismo del Ejercicio 4, aplicado a una relación jerárquica en lugar de una relación de coincidencia: `lego_themes` se une consigo misma, con `h` representando "el hijo" y `p` representando "el padre" del mismo hijo. La condición de unión `h.parent_id = p.id` es lo que convierte una tabla plana en un árbol navegable.

Se usa `INNER JOIN` (no `LEFT JOIN`) porque la pregunta es "cuántos subtemas tiene cada padre *que sí tiene subtemas*": un tema sin hijos simplemente no debe aparecer en el resultado, que es exactamente lo que hace un `JOIN` normal al no encontrar coincidencia.

El podio de piezas de cada año

Sobre **lego**, para cada año entre 2015 y 2017, encuentra los 3 sets con más piezas (el "podio" de ese año). Usa una CTE y una función de ventana.

SOLUCIÓN

```
WITH ranked AS (
  SELECT s.set_num, s.name, s.year, s.num_parts,
         ROW_NUMBER() OVER (PARTITION BY s.year
                             ORDER BY s.num_parts DESC) AS rn
  FROM   lego_sets s
  WHERE  s.year BETWEEN 2015 AND 2017
)
SELECT year, rn, name, num_parts
FROM   ranked
WHERE  rn <= 3
ORDER BY year, rn;
```

year	rn	name	num_parts
2015	1	The SHIELD Helicarrier	2 995
2015	2	Mercedes Benz Arocs 3245	2 792
2015	3	Ferris Wheel	2 463
2016	1	Firehouse Headquarters	4 640
2016	2	Personalised Mosaic Portrait	4 501
2017	1	Assembly Square	4 009

6 de 9 filas mostradas.

POR QUÉ

Este es el patrón "top-N por grupo", uno de los más preguntados en exámenes de SQL. Se usa `ROW_NUMBER()` en lugar de `RANK()` a propósito, a diferencia del Ejercicio 7: aquí se pide exactamente 3 filas por año, sin importar empates, así que se necesita un desempate determinista que garantice siempre 3 y no más.

La CTE es obligatoria por la misma razón que en el Ejercicio 7: el `WHERE rn <= 3` no puede convivir en la misma consulta donde `rn` se calcula, porque el filtro se evalúa antes que la función de ventana en el orden lógico de SQL. Envolver el cálculo en un `WITH` separa las dos fases con claridad.

Buscar empleados por cargo vigente

Sobre **employees**, la consulta ¿quién tiene hoy el cargo 'Senior Engineer'? filtra por `title.title = 'Senior Engineer' AND title.to_date = '9999-01-01'` (la convención de esta base marca los registros vigentes con esa fecha centinela). La tabla tiene 443 308 filas y ningún índice de consulta. Diseña un índice que optimice esta búsqueda y mide el efecto con `EXPLAIN ANALYZE`.

SOLUCIÓN

```
CREATE INDEX idx_title_actual
ON employees.title(title)
INCLUDE (employee_id)
WHERE to_date = '9999-01-01';
ANALYZE employees.title;
```

ANTES – SIN ÍNDICE

Seq Scan on title t	(cost=0.00..9849.62 rows=53308 width=19)
	(actual time=0.011..33.538 rows=85939 loops=1)
Filter: ((title = 'Senior Engineer') AND (to_date = '9999-01-01'))	
Rows Removed by Filter: 357369	
Buffers: shared hit=3192 read=8	
Execution Time: 37.447 ms	

DESPUÉS – ÍNDICE PARCIAL + CUBRIENTE

Index Only Scan using idx_title_actual on title t	
(cost=0.42..1941.82 rows=51130 width=19)	
(actual time=0.035..12.294 rows=85939 loops=1)	
Index Cond: (title = 'Senior Engineer')	
Heap Fetches: 0	
Buffers: shared hit=1 read=426	
Execution Time: 16.114 ms	

POR QUÉ ESTA FORMA DE ÍNDICE Y NO OTRA

Tres decisiones de diseño, cada una justificada por el enunciado:

- 1. Es parcial (WHERE to_date = '9999-01-01').** La condición de vigencia aparece en todas las consultas de este tipo sobre esta tabla. Un índice parcial sólo indexa las filas que cumplen esa condición, así que pesa una fracción de lo que pesaría un índice completo, y sigue sirviendo para exactamente el patrón de consulta que importa.
- 2. Indexa por title, no por employee_id.** La búsqueda entra por el nombre del cargo (una igualdad de baja selectividad relativa: 85 939 de 443 308 filas son vigentes, pero de esas sólo una fracción tiene ese cargo específico), así que esa es la columna que debe encabezar el índice.
- 3. Usa INCLUDE (employee_id).** El SELECT sólo necesita employee_id y title; incluir esa columna dentro del índice permite responder la consulta completa sin tocar la tabla. Por eso el plan final dice Index Only Scan y Heap Fetches: 0: 37,4 ms → 16,1 ms, **2,3× más rápido**, y los bloques leídos bajan de 3 200 a 427.

Plantilla vigente por departamento

Sobre **employees**, calcula cuántos empleados tiene **actualmente** asignados cada departamento (department_employee.to_date = '9999-01-01'). La tabla department_employee tiene 331 603 filas. Diagnostica el plan sin índice, propone uno y comprueba la mejora.

SOLUCIÓN – CONSULTA OBJETIVO

```
SELECT d.dept_name, COUNT(*) AS headcount
FROM employees.department_employee de
JOIN employees.department d ON d.id = de.department_id
WHERE de.to_date = '9999-01-01'
GROUP BY d.dept_name
ORDER BY headcount DESC;
```

dept_name	headcount
Development	61 386
Production	53 304
Sales	37 701
Customer Service	17 569
Research	15 441

5 de 9 filas mostradas.

DIAGNÓSTICO – ANTES (EXTRACTO DEL PLAN)

```
-> Parallel Seq Scan on department_employee de
    Filter: (to_date = '9999-01-01')
    Rows Removed by Filter: 45740
    Buffers: shared read=2160
Execution Time: 116.044 ms
```

```
CREATE INDEX idx_deptemp_actual
ON employees.department_employee(department_id)
WHERE to_date = '9999-01-01';
ANALYZE employees.department_employee;
```

DESPUÉS

```
-> Parallel Index Only Scan using idx_deptemp_actual on department_employee de
    Heap Fetches: 0
    Buffers: shared hit=3 read=204
Execution Time: 96.436 ms
```

POR QUÉ LA MEJORA ES MODESTA (1,2×) Y NO ESPECTACULAR

El índice sí cumplió su función: el acceso a department_employee pasó de Seq Scan con Rows Removed by Filter: 45740 a Index Only Scan con Heap Fetches: 0, y los bloques leídos de ese nodo cayeron de 2 160 a 204 — una reducción del 90 %. Pero el tiempo total de la consulta apenas se mueve, porque después de resolver ese acceso el resto del trabajo (el Hash Join contra department, el HashAggregate sobre 240 mil filas y el Sort final) sigue costando lo mismo.

Esta es la misma lección del taller original aplicada a un caso nuevo: **optimizar el nodo más caro del plan no garantiza que la consulta entera se sienta más rápida** si ese nodo ya no domina el coste total. Verificarlo con EXPLAIN (ANALYZE, BUFFERS) —y no sólo mirar el tiempo final— es lo que permite distinguir "el índice no sirvió" de "el índice funcionó, pero el cuello de botella se movió".

Filtro por género y fecha de nacimiento

Sobre **employees**, se necesita buscar frecuentemente empleados por género y rango de fecha de nacimiento, por ejemplo: hombres nacidos entre enero y junio de 1960. ¿En qué orden deben ir las columnas de un índice compuesto para esta consulta, (gender, birth_date) o (birth_date, gender)? Justifica y demuestra con evidencia.

SOLUCIÓN

```
CREATE INDEX idx_emp_gender_birth
ON employees.employee(gender, birth_date);  -- igualdad primero, rango despues
ANALYZE employees.employee;
```

ANTES – SIN ÍNDICE

```
Parallel Seq Scan on employee
  Filter: ((birth_date >= '1960-01-01') AND (birth_date <= '1960-06-30')
          AND (gender = 'M'))

Rows Removed by Filter: 146558
Buffers: shared read=2504
Execution Time: 36.022 ms
```

DESPUÉS – ÍNDICE (GENDER, BIRTH_DATE)

```
Bitmap Heap Scan on employee
  Recheck Cond: ((gender = 'M') AND (birth_date >= '1960-01-01')
                AND (birth_date <= '1960-06-30'))

Heap Blocks: exact=2315
-> Bitmap Index Scan on idx_emp_gender_birth
    Buffers: shared read=8
Execution Time: 3.943 ms
```

POR QUÉ (GENDER, BIRTH_DATE) Y NO AL REVÉS

La regla general de índices compuestos es: **columnas de igualdad primero, columnas de rango al final**. Un índice B-tree ordena sus entradas de izquierda a derecha, como las páginas de un directorio telefónico ordenado primero por apellido y luego por nombre. Si la primera columna del índice es gender (una igualdad exacta, 'M'), el motor puede saltar directamente al bloque de "M" y desde ahí leer un rango contiguo y ordenado de fechas de nacimiento.

Si el orden fuera (birth_date, gender), la condición de rango sobre birth_date quedaría primero: el índice podría acotar las fechas, pero dentro de ese rango las filas de género 'M' y 'F' estarían entremezcladas, obligando a revisar más entradas del índice de las necesarias.

Resultado medido: 36,02 ms → 3,94 ms, **9,1× más rápido**, con 6 907 filas que cumplen ambas condiciones sobre 300 024 empleados (2,3 % de selectividad, dentro del rango donde un índice conviene).

Diagnostica este plan sin ejecutar nada

Un compañero ejecutó esta consulta sobre payment (14 596 filas) y obtuvo este plan. Sin ejecutar nada tú mismo: **(a)** identifica qué indica que la consulta tiene un problema, **(b)** propone el índice que lo resolvería y **(c)** predice qué cambiaría en el plan.

```
Seq Scan on payment  (cost=0.00..346.94 rows=340 width=20)
    (actual time=0.020..1.371 rows=340 loops=1)
    Filter: ((amount >= 9.99) AND (amount <= 11.99))
    Rows Removed by Filter: 14256
    Buffers: shared hit=128
Execution Time: 1.423 ms
```

SOLUCIÓN

(a) Diagnóstico. La línea clave es Rows Removed by Filter: 14256: de 14 596 filas leídas, el motor descartó 14 256 (el 97,7 %) después de traerlas a memoria. El Seq Scan confirma que no existe ningún índice sobre amount; el motor no tuvo más remedio que leer la tabla entera fila por fila.

(b) Índice propuesto.

```
CREATE INDEX idx_payment_amount ON payment(amount);
ANALYZE payment;
```

(c) Predicción y verificación. Con un índice B-tree sobre amount, el motor puede localizar directamente el rango [9.99, 11.99] sin leer las filas descartadas. Se espera un cambio de Seq Scan a Bitmap Heap Scan (o Index Scan), la desaparición de la línea Rows Removed by Filter, y una caída en Buffers. La ejecución real lo confirma:

```
Bitmap Heap Scan on payment  (cost=7.79..140.92 rows=342 width=20)
    (actual time=0.052..0.171 rows=340 loops=1)
    Recheck Cond: ((amount >= 9.99) AND (amount <= 11.99))
    Heap Blocks: exact=102
    Buffers: shared hit=104 read=1
    -> Bitmap Index Scan on idx_payment_amount
        Buffers: shared hit=2 read=1
Execution Time: 0.204 ms
```

POR QUÉ ESTE EJERCICIO ES DISTINTO A LOS ANTERIORES

Aquí no se pide escribir SQL sino **leer** un plan de ejecución, que es la habilidad que distingue a alguien que memorizó CREATE INDEX de alguien que entiende cuándo usarlo. En un examen extenso, este formato —plan dado, diagnóstico pedido— es tan probable como el formato inverso (consulta dada, plan pedido), y suele aparecer justo cuando el resto del examen ya evaluó la sintaxis de creación de índices.

Las tres señales que hay que saber leer de memoria son: Seq Scan sobre una tabla grande con filtro selectivo (síntoma), Rows Removed by Filter alto (cuantifica el desperdicio) y Buffers alto en proporción al tamaño esperado del resultado (mide el costo real de E/S).

¿Cuándo NO crear un índice?

Se te presentan cuatro escenarios sobre la base **employees**. Para cada uno, decide si crear el índice propuesto es una buena idea y justifica con los conceptos del taller (selectividad, tamaño de tabla, frecuencia de uso, costo de escritura).

#	Escenario	Índice propuesto
a	Reporte anual que filtra employee.hire_date entre dos fechas (devuelve ~2 % de la tabla), ejecutado una vez al año.	CREATE INDEX ON employee(hire_date)
b	Dashboard en vivo que cada segundo consulta WHERE gender = 'M' sobre employee (devuelve ~54 % de la tabla).	CREATE INDEX ON employee(gender)
c	department tiene 9 filas. Un informe filtra WHERE dept_name = 'Sales' varias veces al día.	CREATE INDEX ON department(dept_name)
d	salary recibe un INSERT nuevo por cada aumento salarial (alta tasa de escritura) y rara vez se consulta por amount exacto.	CREATE INDEX ON salary(amount)

SOLUCIÓN

(a) Sí, crear el índice — aunque se use poco. La selectividad (2 %) es exactamente el rango donde un índice B-tree rinde, tal como se demostró en el Ejercicio 14 y en el taller original (7× y 37,5× de mejora con selectividades similares). Que se ejecute una vez al año no es motivo para no indexar: el costo de mantenimiento de un índice sobre una tabla de 300 mil filas es bajo, y cuando se ejecuta, la diferencia entre segundos y milisegundos sí importa para quien espera el reporte.

(b) No crear el índice. Un filtro que devuelve el 54 % de la tabla no es selectivo: como se vio en el taller (Fase A6, filtro con 81 % de las filas), el planificador simplemente ignora un índice así y hace un Seq Scan de todas formas, porque saltar entre índice y tabla para más de la mitad de las filas es más lento que leer todo de corrido. Crear este índice sólo añadiría peso muerto.

(c) Probablemente no vale la pena, aunque no hace daño. Con 9 filas, department cabe casi entera en una sola página de disco: un Seq Scan ya es tan barato como puede ser un acceso por índice, quizás más barato. Este es el mismo caso del idx_sets_year del taller original, donde el índice funcionó (5,1×) pero sobre una consulta que ya tardaba menos de un milisegundo: la mejora es real pero irrelevante en la práctica.

(d) No crear el índice. Cada índice adicional tiene que actualizarse en cada INSERT, UPDATE o DELETE sobre esa tabla. Si salary recibe escrituras frecuentes y ese índice apenas se usaría en lecturas, el costo de mantenerlo (cada escritura ahora también reescribe el índice) supera el beneficio. Ésta es la otra cara de la moneda que muestran los tamaños del taller: employees.salary ya tiene 86 MB de índices sobre 142 MB de datos —un 60 % de sobrecosto— antes de añadir ninguno nuevo.

POR QUÉ ESTE EJERCICIO CIERRA EL BANCO

Los catorce ejercicios anteriores entrenan "cómo escribir la consulta" y "cómo crear el índice correcto". Este entrena la pregunta que normalmente decide una nota de examen: **saber cuándo la respuesta correcta es no hacer nada**. Un índice mal puesto no es neutro — cuesta espacio en disco y ralentiza cada escritura futura — así que la pregunta "¿debería crear un índice aquí?" tiene tanto peso como "¿qué índice debería crear?".